

Distributed Systems - 17_12

King algorithm

It is an adaptation of hierarchical consensus for byzantine failures. There we have leaders for different rounds, and also here, we'll have leaders that rotate.

The assumption is that the system is synchronous, and $f < \frac{N}{3}$. It uses authenticated channels and not signatures, so we can't do better than this.

It runs in $f + 1$ phases. Each phase is a time slot composed by three synchronous rounds.

So, for f_0 (phase 0), we'll have r_0, r_1, r_2 , and so on and so forth.

In each of this phase, we'll have a different king. For simplicity, we assume the leader of phase i will be p_i .

Each node will keep a variable x which, at the beginning, is equal to our proposal.

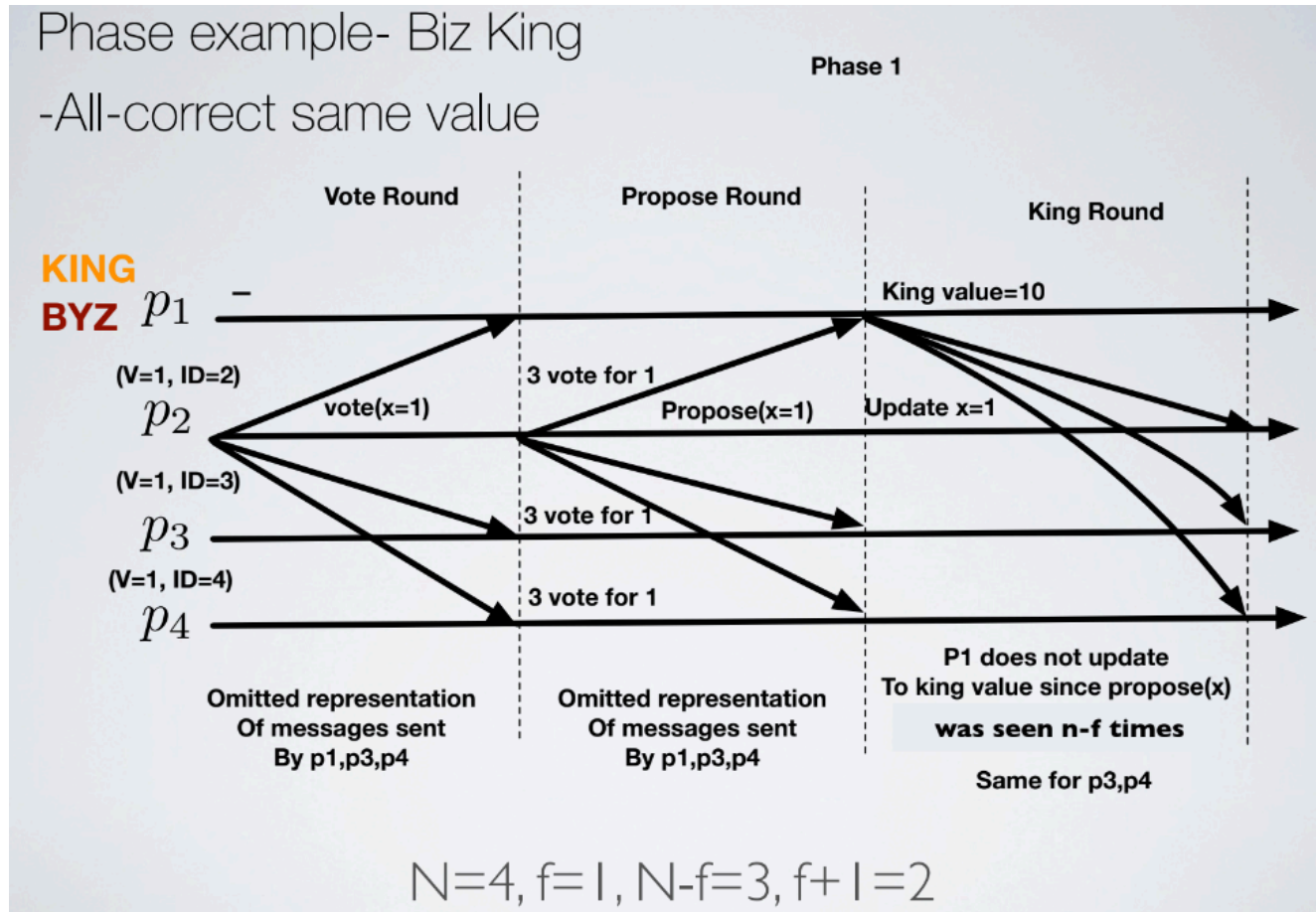
At the end of phase $f + 1$ we'll decide value x .

By having $f + 1$ phases, we'll have at least one correct process among the set of kings.

Our phase f_0 is divided in three rounds. The rounds are:

1. Vote round: In the vote round, we broadcast our value to everyone (including myself obviously). Since the system is synchronous, by the end of the round I will receive the vote from everyone who's correct.
2. Propose round: Once the values are collected, we count the number of votes for a value. If I see a value proposed $N - f$ times, then I will send this value to everyone. At the end of the round, if we see a message proposed $f + 1$ times, we'll change our proposal to this new one.
3. King round: In this round, the king (for example p_0 in f_0), will broadcast this value to everyone in the system. A node will change its idea to the one of the king $\iff$ in the propose round, we've seen no value reaching the threshold $N - f$, otherwise we discard it.

Let's take a look at an execution:

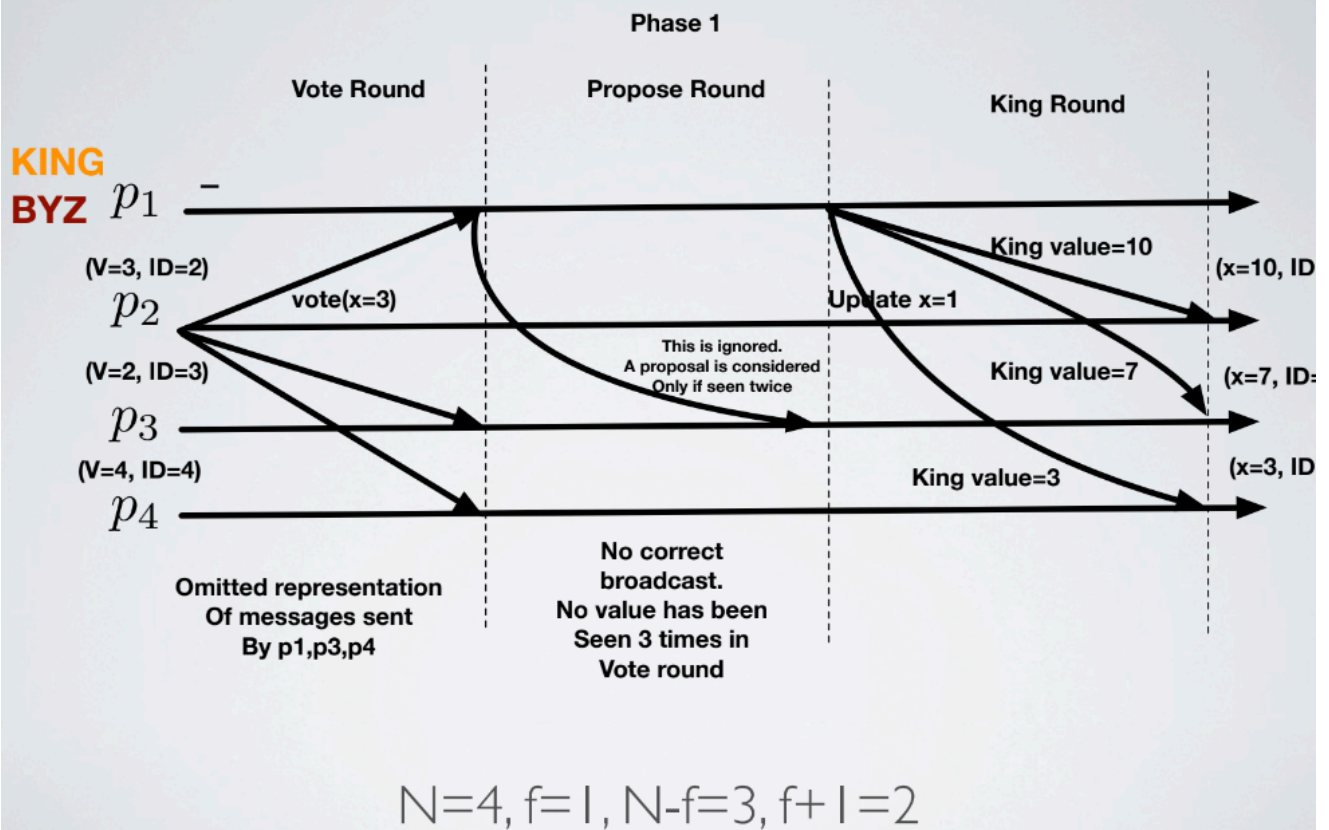


At the end of the vote round, for example, p_3 will see three votes for 1.

Considering that $3 = N - f$, then I will propose this value to everyone. At the end of the propose round, the processes will see the value $N - f$ times, which also means that we'll discard the value sent by the leader.

Now, let's take a look at an example where different corrects start with different values.

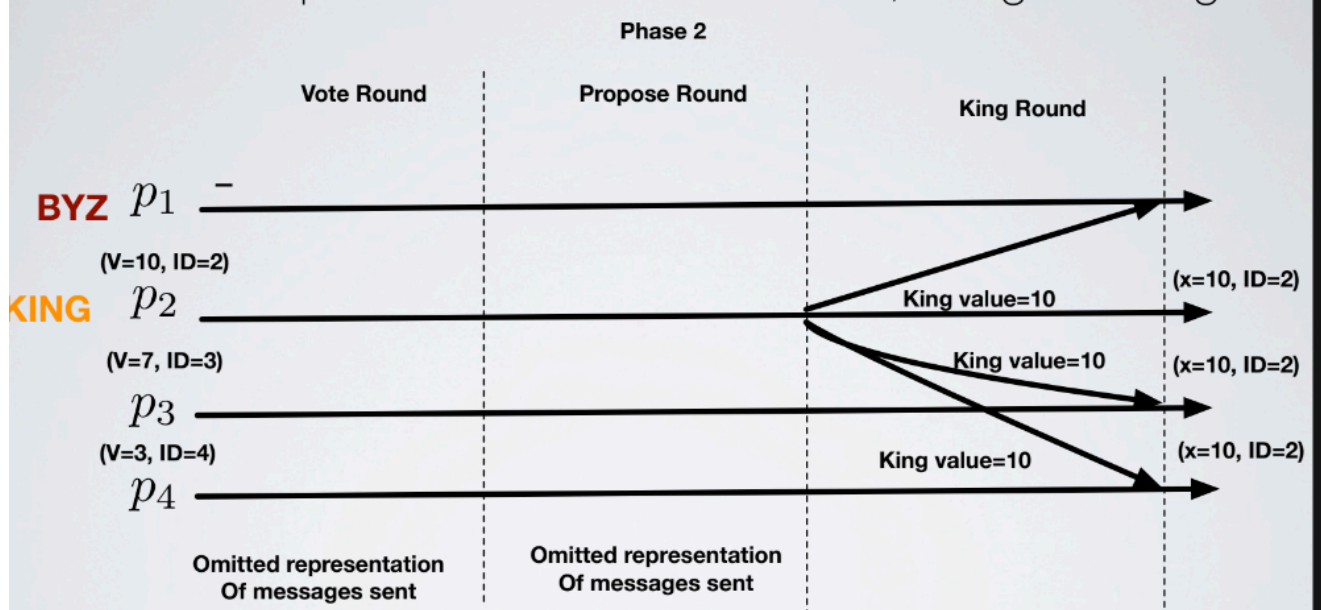
Phase example Biz King - not all correct same values



At the end of the vote round, we could see two votes, as the byzantine process could vote for a value as well. Because no votes reach the threshold of $N - f$, no one will propose a value.

If we have a byzantine king, it could keep having processes having different values. In phase 2, we'll have p_2 as the king, and it's a correct process.

Phase example- not all correct same val, but good King



If a king is honest, it forces agreement (if not already reached).

Essentially the king algorithm is based on two mechanisms:

- Mechanism 1 - All same no change of idea (Implemented in Vote-Propose rounds): If all correct processes have the same value for x , they detect this during Vote-Propose round and they do not get influenced by a byzking and other byzantines.
- Mechanism 2 - Not All Same: Correct King imposes value (implemented in the king round) -> if corrects disagree the correct king imposes agreement.

Let's take a look at the code:

Algorithm 17.14 King Algorithm (for $f < n/3$)

```
1:  $x = \text{my input value}$ 
2: for phase = 1 to  $f + 1$  do
    Vote
3:   Broadcast  $\text{value}(x)$ 
    Propose
4:   if some  $\text{value}(y)$  received at least  $n - f$  times then
5:     Broadcast  $\text{propose}(y)$ 
6:   end if
7:   if some  $\text{propose}(z)$  received more than  $f$  times then
8:      $x = z$ 
9:   end if
    King
10:  Let node  $v_i$  be the predefined king of this phase  $i$ 
11:  The king  $v_i$  broadcasts its current value  $w$ 
12:  if received strictly less than  $n - f$   $\text{propose}(y)$  then
13:     $x = w$ 
14:  end if
15: end for
```

At the end of phase $f+1$ each correct decides value x .

Since the system is synchronous, we can imagine processes are synchronised with the start of the round. So everyone will start executing the same line together. At first we'll have everyone do a Broadcast, and everyone will receive a message from a process if it's correct.

Then we switch to the propose phase, and there we check if a value has been

received $N - f$ times, and if yes we send our proposal to everyone.

Then, at the end of the round, if a proposal has been received $f + 1$ times, we change idea.

Then, we go to the king phase, and a process will change idea only if it didn't receive $N - f$ proposes for a value.

We have to show this algorithm solves the byzantine agreement problem, by proving the properties are implemented.

We have that termination is immediate, as every correct will execute every line of the for $f + 1$ times, and thus it will conclude.

As for integrity, it's also trivial. After the for we have a single line that is decide x , so we're ok as we can't execute it twice.

For validity: It's based on a theorem: if all correct starts with the same value, then that value is decided. Let's start with all corrects starting with the same value, then everyone will broadcast that value on line 3. Then every correct will see at least $N - f$ votes for that value. Then every correct will execute line 5, and every correct will start receiving the propose. Line 7 will be executed by all correct processes, and every correct will change opinion to their own decision (so basically they won't).

Then we have the king. We don't care if the king is byzantine or not, as everyone has received $N - f$ proposal for a value, so the predicate at line 12 will be false for everyone.

I can show this happens with $f + 1$ phases.

Now, we need some Lemma.

If a correct process p proposes x in a phase, then it's not possible there is another correct p' proposing $y \neq x$. When is that we propose a value? We need to see $N - f$ votes for that to happen for a value. Let's suppose p proposes x (it's correct). If p is proposing x , then he has seen x $N - f$ times. Notice that $N - f > \frac{N-f}{2}$. This means there are at least k processes voting for x . Now let's take p' voting for y . If p' is proposing y , then he has seen $N - f$ votes for that value. Then we've received k votes for y . Then we have an intersection of $f + 1$ size (a byzantine quorum has an intersection of $f + 1$, in contrast to a normal quorum that has size 1). Then, by this, we have that the lemma is true.

There is also Lemma 2. There is a phase in which we have a correct king (trivial).

Observation 1: only a single value can be proposed more than $f + 1$ times.

Suppose there are $f + 1$ processes proposing x , we know that at least one is correct. Now suppose we also have $f + 1$ processes proposing y , then it's a contradiction by Lemma 1.

We have now lemma 3: after a phase with correct king, all correct processes have

the same value. The correct king send the same value to each process. All correct that change their proposal to the one of the king, get the same value. The problem is that there is a subset of corrects rejecting the value of the king (by reaching the threshold of $N - f$ for proposed values). We have to show that for processes that don't accept values from the king, have in memory the value of the king.

Remember we have a threshold mechanism.

Continue the proof from slides.

The king algorithm respects the agreement property, by Lemma 3 and Lemma 2.

What is the message complexity? It's n^2 .

The message delay? It's $3 \cdot (f + 1)$.

Exercises notes:

Remember that in synchronous systems, if we don't know the local computation step time, we can't say that after time t if we didn't receive an answer, the process has crashed. We need to considerate $t + \Delta$, with Δ being the time necessary to compute the local step.

Remember that the happened before relationship has closure, so if we have $m_2 \rightarrow m_3$ and $m_3 \rightarrow m_4$, then we also have to specify $m_2 \rightarrow m_4$.